L1-2 讲义：大语言模型基础

课程: MBA 大模型智能体课程
课时: 第 1-2 讲 (共 3 小时)
适用: 讲师备课 / 学生预习与复习

目录

1. 什么是大语言模型?
2. LLM 核心能力
3. 主流模型对比
4. 从 Prompt Engineering 到 Context Engineering □
5. API 调用实践
6. 成本与选型
7. 参考资料

---

1. 什么是大语言模型?

1.1 定义

大语言模型 (Large Language Model, LLM) 是基于 Transformer 架构、在海量文本数据上训练的神经网络模型，能够理解和生成人类语言。

1.2 发展历程

年份	里程碑	意义
2017	Transformer 发布	注意力机制革命
2018	GPT-1, BERT	预训练 + 微调范式
2020	GPT-3 (175B)	涌现能力出现
2022	ChatGPT	AI 大众化元年
2023	GPT-4, Claude 2	多模态、长上下文
2024	Claude 3.5, GPT-4o	推理能力飞跃
2025	o3, Claude 4	Agent 时代开启

1.3 规模与涌现

```
模型参数规模增长：
GPT-2 (1.5B) → GPT-3 (175B) → GPT-4 (估计1.8T) → ...
                    ↓
                涌现能力出现
                - 推理
                - 代码生成
                - 多步骤规划
```

涌现能力 (Emergent Abilities): 小模型没有、大模型突然具备的能力

1.4 工作原理简述—MBA 需要知道的程度

作为 MBA 学生，你不需要成为 AI 工程师，但需要理解几个核心概念才能做出正确的商业决策。

什么是 Token？（这决定了你的成本） **商业直觉**：Token 是 AI 的"计费单位"，就像电话按分钟计费、打车按公里计费一样，LLM 按 token 计费。

技术定义：Token 是模型处理文本的最小单位，大致相当于：- 英文：1 个单词 ≈ 1.3 tokens - 中文：1 个汉字 ≈ 1.5-2 tokens

为什么 MBA 必须懂 Token：

场景	Token 数	GPT-4 成本	商业启示
一封邮件	~200	$0.006	几乎免费
一份 10 页报告	~3,000	$0.09	仍然很便宜
1000 封客服对话	~500,000	$15	比雇人便宜 100 倍
分析一本书	~100,000	$3	人工需要一周

决策启示：- 短文本任务（分类、提取）→ 成本可忽略 - 长文本任务（文档分析）→ 需要计算 ROI - 高频任务 → Token 用量是关键成本因素

什么是 Context Window？（这决定了 AI 能"记住"多少） **商业直觉**：Context Window 就是 AI 的"工作记忆"容量。想象一位助理：- 有的助理只能同时处理 3 页文件（小窗口）- 有的助理能同时处理 300 页文件（大窗口）

技术定义：模型单次能处理的最大 token 数。

2026 年主流模型 Context Window：

模型	Context Window	等于多少页文档	商业意义
GPT-4	128K tokens	~200 页	可以分析一份完整的商业计划书
Claude 4	200K tokens	~300 页	可以分析多份合同 + 历史邮件
Gemini 2	1M tokens	~1500 页	可以分析整本书 + 相关资料

为什么这对产品经理很重要：- 窗口太小 → 需要"分块"处理，丢失全局视角 - 窗口足够大 → 一次性给出完整上下文，回答更准确 - 但：更长的 context = 更高的成本 + 更慢的响应

决策框架：

```
任务复杂度低 + 信息量小 → 选便宜的小窗口模型
任务复杂度高 + 需要全局视角 → 选大窗口模型
成本敏感 + 信息量大 → 先RAG检索，再用小窗口
```

LLM 的"思考"过程（简化版）

```
用户输入 → 切分成Token → 转成数字(Embedding) → 模型计算 → 输出概率 → 选择最可能的下一个词

           计费起点          向量空间                 这里消耗算力
```

核心机制（知道名词即可）：- **自注意力 (Self-Attention)**: 让模型理解"他"指代谁、"这个"是什么 - **自回归生成**: 一个词一个词地写，像人打字一样

---

2. LLM 核心能力

2.1 能力矩阵—什么任务适合交给 AI？

作为管理者，你需要知道哪些任务可以放心交给 AI，哪些需要人工把关。

能力	AI 擅长程度	典型商业应用	是否需要人工审核
文本理解	□ □ □ □ □	邮件分类、情感分析、合同要点提取	低风险场景可自动化
内容生成	□ □ □ □	营销文案、报告初稿、邮件回复	建议人工审核后发布
翻译	□ □ □ □ □	多语言客服、文档本地化	专业领域需审核
数据分析	□ □ □ □	报表解读、趋势识别	关键决策需复核
代码编写	□ □ □ □ □	脚本开发、代码审查	需测试验证
精确计算	□ □	财务计算、统计分析	□ 必须用工具验证
实时信息	□	股价、新闻、天气	□ 必须用工具获取

MBA 关键洞察：- AI 最强的是"语言任务"（理解、生成、转换）- AI 最弱的是"精确任务"（数学、实时数据）- 聪明的做法：**让 AI 做语言，让工具做计算**

2.2 In-Context Learning —为什么 AI 可以"即学即用"

商业场景：假设你需要 AI 帮你处理一种特殊格式的报告，传统软件需要程序员开发几周，但 LLM 只需要你"示范"几个例子。

原理：LLM 可以通过 Prompt 中的示例"学会"新任务，无需重新训练。

你的示范：
```
输入："Q1收入1.2亿" → 输出：{"quarter": "Q1", "revenue": 120000000}
输入："Q2收入1.5亿" → 输出：{"quarter": "Q2", "revenue": 150000000}

新输入："Q3收入1.8亿" → AI自动输出：{"quarter": "Q3", "revenue": 180000000}
```

三种学习模式：

模式	示例数	效果	适用场景
Zero-shot	0 个	依赖模型自身能力	常见任务
One-shot	1 个	明确格式和风格	格式要求明确
Few-shot	2-5 个	稳定可靠	复杂/专业任务

商业启示：- 不需要请 AI 公司定制模型 - 只需要准备好示例数据 - **你的行业知识 + 通用 AI = 专业 AI 助手**

2.3 推理能力—如何让 AI"想清楚再说"

商业问题：你让 AI 分析一份复杂的财务数据，它直接给出结论，但你不知道它是怎么得出的。更糟的是，有时候它会"自信地犯错"。

解决方案：Chain-of-Thought (CoT)

让 AI"展示思考过程"，就像让员工汇报时"说说你是怎么想的"：

```
  直接问："我们应该进入这个市场吗？"
→ AI直接说"应该"或"不应该"，你无法判断对错

  CoT问法："分析这个市场机会，请从以下角度逐步思考：
    1. 市场规模和增长率
    2. 竞争格局
    3. 我们的竞争优势
    4. 进入成本和风险
    最后给出建议。"
→ AI展示完整分析链条，你可以检验每一步
```

为什么 MBA 需要知道 CoT：- 提高 AI 输出的可审计性 - 减少"幻觉"错误 - 让 AI 的分析过程可被质疑和改进

2.4 能力边界—知道 AI"不能做什么"比"能做什么"更重要

AI 强项（放心用）	AI 弱项（需要工具/人工）	商业风险
语言理解与生成	☐ 精确数学计算	财务错误
模式识别与类比	☐ 实时信息获取	信息过时
创意与写作	☐ 长期记忆保持	前后矛盾
代码生成与解释	☐ 物理世界交互	执行错误
知识综合与解释	☐ 专业判断（法律/医疗）	合规风险

关键认知（避免踩坑）：1. **AI 会编造数据**—永远不要直接引用 AI 给的统计数字 2. **AI 知识有截止日期**—问最近的事它可能不知道 3. **AI 没有"常识"判断**—它不知道某些建议在现实中行不通 4. **AI 会"自信地犯错"**—语气越自信不代表越准确

最佳实践：- 重要数字 → 用搜索工具验证 - 关键决策 → 人工最终审核 - 专业领域 → 结合领域专家

---

3. 主流模型对比—如何为你的业务选择合适的模型

3.1 2026 年主流模型

模型	厂商	特点	适用场景
GPT-5	OpenAI	综合最强、多模态	通用任务
Claude 4	Anthropic	长上下文、安全	企业应用
Gemini 2	Google	原生多模态	搜索、视觉
Llama 3	Meta	开源、可本地部署	定制化需求
Qwen 2.5	阿里	中文优化	国内场景
DeepSeek R2	DeepSeek	推理强、成本低	性价比场景

3.2 选型考量

维度	考量因素
能力	任务匹配度、推理能力
成本	输入/输出 Token 价格
延迟	响应速度要求
上下文	需要处理的文本长度
合规	数据隐私、本地部署需求
生态	API 稳定性、工具支持

3.3 成本对比 (2026 年参考价格)

模型	输入 $(/1Mtokens)$| (/1M tokens)	
GPT-5	$15	$60
Claude 4 Opus	$15	$75
Claude 4 Sonnet	$3	$15
Gemini 2 Pro	$7	$21
DeepSeek R2	$0.5	$2

---

4. 从 Prompt Engineering 到 Context Engineering □

4.1 概念演进

□ 来源: Anthropic - Effective Context Engineering for AI Agents (2025.9.29)

Anthropic 的观点：> "At Anthropic, we view context engineering as the natural progression of prompt engineering."

阶段	关注点	核心问题
Prompt Engineering	如何写好提示词	"怎么问才能得到好答案?"
Context Engineering	如何管理整个上下文	"什么信息组合最能引导模型行为?"

定义：- **Context** = 采样时传递给 LLM 的所有 token - **Context Engineering** = 策展和维护最优 token 组合的策略，包括 prompt 之外的所有信息

Karpathy 的定义：> "Context engineering is the art and science of curating what will go into the limited context window from that constantly evolving universe of possible information."

4.2 为什么 Context Engineering 很重要?

注意力预算 (Attention Budget)：

Anthropic 观察到：> "LLMs, like humans, lose focus or experience confusion at a certain point."

Context Rot 现象：上下文 token 越多，模型准确召回信息的能力越弱。

原因：- Transformer 的 n^2 复杂度—n 个 token 需要 n^2 对关系 - 训练数据中短序列更常见，模型对长上下文依赖经验不足 - 即使 128K 窗口也不够用，边际效益递减

结论：Context 必须被视为**有限资源**，需要精心策展。

4.3 有效 Context 的组成

```
                完整Context

  1. System Prompt（系统指令）
  2. Tools（工具定义 + MCP）
  3. Examples（Few-shot示例）
  4. Retrieved Data（检索数据）
  5. Message History（历史消息）
  6. Current Input（当前输入）
```

核心原则 (Anthropic)：> "Good context engineering means finding the smallest possible set of high-signal tokens that maximize the likelihood of some desired outcome."

4.4 System Prompt 的"最优高度"

Anthropic 指出两种失败模式：

```
过于具体            最优高度            过于模糊
   ↓                   ↓                   ↓
硬编码if-else逻辑     强启发式指导         缺乏具体信号
脆弱、难维护         灵活、可适应         输出不可控
```

最佳实践：- 使用 XML 标签或 Markdown 分节：`<instructions>`, `## Tool guidance` - **追求最小完备集**：minimal 但不一定 short - 先用最简 prompt + 最强模型测试，基于失败 case 迭代

4.5 工具设计原则

Anthropic 建议：> "One of the most common failure modes we see is bloated tool sets that cover too much functionality or lead to ambiguous decision points."

- 工具应**自包含、健壮、描述清晰**
- 如果人类工程师都分不清用哪个工具，Agent 更不行
- 工具返回应 **token 高效**

4.6 Context 检索策略

从预计算 → 即时检索 (Just-in-Time)：

策略	说明	适用场景
预计算检索	预先 embedding，推理前召回	静态知识库
即时检索	保留轻量引用，运行时动态加载	动态环境、代码库
混合策略	预加载核心 + 按需探索	通用场景

Claude Code 的做法：- `CLAUDE.md` 预加载到 context - 其他文件通过 `glob/grep` 按需获取 - 避免陈旧索引问题

4.7 长时间任务的 Context 管理

对于超出 context 窗口的长任务，Anthropic 提出三种技术：

(1) Compaction (压缩)

`对话接近窗口上限 → 摘要压缩 → 用摘要重新初始化context`

- 保留架构决策、未解决 bug、实现细节
- 丢弃冗余工具输出
- 继续时加载压缩摘要 + 最近 5 个文件

注意：过度压缩可能丢失关键细节

(2) Structured Note-taking (结构化笔记) Agent 定期写笔记到外部存储，稍后拉回 context。

案例：Claude playing Pokémon - 跨数千步追踪目标："过去 1234 步我在 Route 1 训练，Pikachu 升了 8 级…"- 自发发展地图、策略笔记 - Context 重置后读取笔记继续

(3) Sub-agent Architectures (子 Agent 架构)

```
主Agent（高层计划）
      子Agent A（深度探索，数万token)→ 返回1-2K摘要
      子Agent B（专项任务）→ 返回摘要
      子Agent C ...
```

- 关注点分离：详细搜索隔离在子 Agent
- 主 Agent 专注综合分析

选择依据：- Compaction → 需要大量来回对话 - Note-taking → 有明确里程碑的迭代开发 - Multi-agent → 复杂研究、并行探索

4.8 Manus 的实战经验

> ☐ 来源: Manus - Context Engineering for AI Agents (2025.7.18)

Manus 团队重建了 4 次 Agent 框架，总结的核心经验：

(1) 围绕 KV-Cache 设计—为什么这对成本至关重要？　什么是 KV-Cache？（MBA 友好解释）

想象你是一位 CEO，每天有 100 封邮件需要处理。两种工作方式：

方式 A：无缓存（低效） - 每封邮件你都从头阅读公司背景、项目历史、人员关系 - 即使是同一项目的第 10 封邮件，你还是要重新理解所有背景 - 结果：每封邮件花 30 分钟

方式 B：有缓存（高效） - 第一封邮件：花 30 分钟理解背景，但把理解"缓存"在脑中 - 后续邮件：直接调用已有理解，只需 5 分钟处理新内容 - 结果：效率提升 6 倍

KV-Cache 就是 LLM 的"记忆缓存"：- K (Key) 和 V (Value) 是模型处理过的中间计算结果 - 如果前面的对话没变，这些计算可以复用 - 如果前面的对话变了，所有计算要重新做

商业影响（这才是 MBA 关心的）：

场景	Claude Sonnet 成本	差距
缓存命中	$0.30/百万 token	基准
缓存未命中	$3.00/百万 token	**10 倍！**

真实案例：- 一个 Agent 每天处理 1000 次对话 - 每次对话 10K tokens - 缓存命中：每天成本 $3 - 缓存未命中：每天成本 $30 - **一年差距：$10,000 vs $1,000**

产品经理/创业者必须理解的规则：

```
保持Prompt前缀稳定
  → 系统提示不要每次都变

Context只追加，不修改历史
  → 对话记录只往后加，不改前面的

序列化保持确定性（JSON key顺序固定）
  → 技术细节，但要告诉工程师

系统提示开头放精确到秒的时间戳
  → 这会让每次调用都"缓存未命中"

中途修改之前的action/observation
  → 改了就要全部重算
```

一句话总结：KV-Cache 是 LLM 版的"会议纪要"——如果你每次开会都从头介绍项目背景，效率低且成本高；聪明的做法是复用之前的理解。

(2) 遮蔽而非移除 (Mask, Don't Remove)　动态添加/移除工具会：- 破坏 KV-cache (工具定义在 context 前部) - 让模型困惑 (历史引用了不存在的工具)

解决方案：用状态机 + logits masking 控制可用工具

(3) 文件系统即 Context　128K tokens 仍然不够 → 像人类一样用外部存储

```
不要：把所有数据塞进context
要：　保留轻量引用（文件路径、查询）
      需要时动态加载
```

4.9 Prompt Engineering 基础 (仍然重要!)

虽然进化到 Context Engineering，Prompt Engineering 的基础技巧仍是核心：

Prompt 结构：

[系统指令] - 定义角色和行为规范
[上下文] - 提供背景信息
[示例] - Few-shot示例
[用户输入] - 具体问题或任务
[输出格式] - 期望的输出结构

核心技巧：| 技巧 | 说明 | 示例 | |——|——|——| | **角色设定** | 赋予模型专家身份 | “你是一位资深投资分析师…” | | **明确指令** | 清晰描述任务 | “分析以下财报，列出 3 个关键风险” | | **格式约束** | 指定输出格式 | “以 JSON 格式输出…” | | **分步引导** | 要求逐步思考 | “请按以下步骤分析：1. 2. 3.” | | **负面约束** | 说明不要做什么 | “不要编造数据，不确定时说明” |

高级技巧：- **Chain-of-Thought (CoT)**: 引导逐步推理 - **Self-Consistency**: 多角度分析后综合 - **ReAct**: 思考-行动-观察循环

4.10 总结：从单次调用到 Agent

维度	单次调用 (Prompt)	Agent (Context)
关注点	写好这一条指令	管理整个信息流
时间范围	一次交互	多轮、长时间运行
信息来源	用户输入	用户 + 工具 + 记忆 + 检索
优化目标	输出质量	输出质量 + 效率 + 成本
核心技能	写 Prompt	策展 Context

Anthropic 的结论：> “Context engineering represents a fundamental shift in how we build with LLMs…treating context as a precious, finite resource will remain central to building reliable, effective agents.”

---

5. API 调用实践

5.1 OpenAI API 示例

```python
from openai import OpenAI

client = OpenAI(api_key="your-api-key")

response = client.chat.completions.create(
    model="gpt-4",
    messages=[
        {"role": "system", "content": " 你是一位专业的商业分析师"},
        {"role": "user", "content": " 分析特斯拉 2024 年的竞争优势"}
    ],
    temperature=0.7,
    max_tokens=1000
)

print(response.choices[0].message.content)
```

5.2 关键参数

参数	说明	建议值
temperature	随机性 (0-2)	分析任务 0.3，创意任务 0.8
max_tokens	最大输出长度	根据任务设置

参数	说明	建议值
top_p	核采样	通常 0.9-1.0
frequency_penalty	重复惩罚	0-0.5
presence_penalty	话题新颖性	0-0.5

5.3 Streaming 响应

```
stream = client.chat.completions.create(
    model="gpt-4",
    messages=[...],
    stream=True
)

for chunk in stream:
    if chunk.choices[0].delta.content:
        print(chunk.choices[0].delta.content, end="")
```

5.4 错误处理

```
import openai
from tenacity import retry, stop_after_attempt, wait_exponential

@retry(stop=stop_after_attempt(3), wait=wait_exponential(multiplier=1, min=4, max=10))
def call_llm(prompt):
    try:
        response = client.chat.completions.create(...)
        return response
    except openai.RateLimitError:
        print("Rate limited, retrying...")
        raise
    except openai.APIError as e:
        print(f"API error: {e}")
        raise
```

---

6. 成本与选型

6.1 成本计算

```
总成本 = (输入tokens × 输入单价) + (输出tokens × 输出单价)

示例: 分析一份10页报告 (约5000 tokens输入), 生成1000字分析 (约1500 tokens输出)
GPT-4成本 = 5000 × $0.03/1K + 1500 × $0.06/1K = $0.15 + $0.09 = $0.24
```

6.2 成本优化策略

策略	说明	效果
Prompt 压缩	精简输入，去除冗余	减少 30-50% 输入成本
模型降级	简单任务用小模型	成本降低 5-10 倍
缓存	相同查询复用结果	避免重复调用
批处理	合并请求	减少 API 开销
本地模型	部署开源模型	边际成本趋近于零

6.3 选型决策树

```
任务复杂度?
  简单（分类/提取）→ 小模型（GPT-3.5/Claude Haiku）
  中等（分析/生成）→ 中等模型（GPT-4o-mini/Claude Sonnet）
  复杂（推理/创作）→ 大模型（GPT-4/Claude Opus）

数据敏感度?
  高敏感 → 本地部署（Llama/Qwen）
  可接受 → 云API

延迟要求?
  实时（<1s）→ 小模型 + 流式
  容忍（>3s）→ 大模型
```

---

7. 参考资料

官方文档

- OpenAI API Documentation
- Anthropic Claude Documentation
- Google Gemini Documentation

Context Engineering ☐ 新增

- Anthropic: Effective Context Engineering for AI Agents - 官方指南
- Manus: Context Engineering for AI Agents - 实战经验
- Karpathy on Context Engineering - 概念定义
- Context Engineering Guide - 系统教程

Prompt Engineering

- Prompt Engineering Guide
- OpenAI Cookbook
- Anthropic Prompt Engineering Docs

视频

- Andrej Karpathy: Intro to LLMs
- Andrej Karpathy: Deep Dive into LLMs

论文

- Attention Is All You Need - Transformer 原论文
- Language Models are Few-Shot Learners - GPT-3
- Chain-of-Thought Prompting

课后实践

1. 注册 OpenAI/Anthropic API，完成第一次调用
2. 设计一个业务场景的 Prompt，迭代优化 3 次
3. 计算你业务场景的月度 API 成本估算

本讲义最后更新：2026-02-28